

PEAP-AI Technical Summary

This document summarizes the workflow, architecture, programming stack, and LLM usage of PEAP-AI as implemented in the LLED_bot_MVP repository (LLED 200 Essay Feedback MVP) for formative AI feedback on Descriptive Report drafts by Manshan Lin.

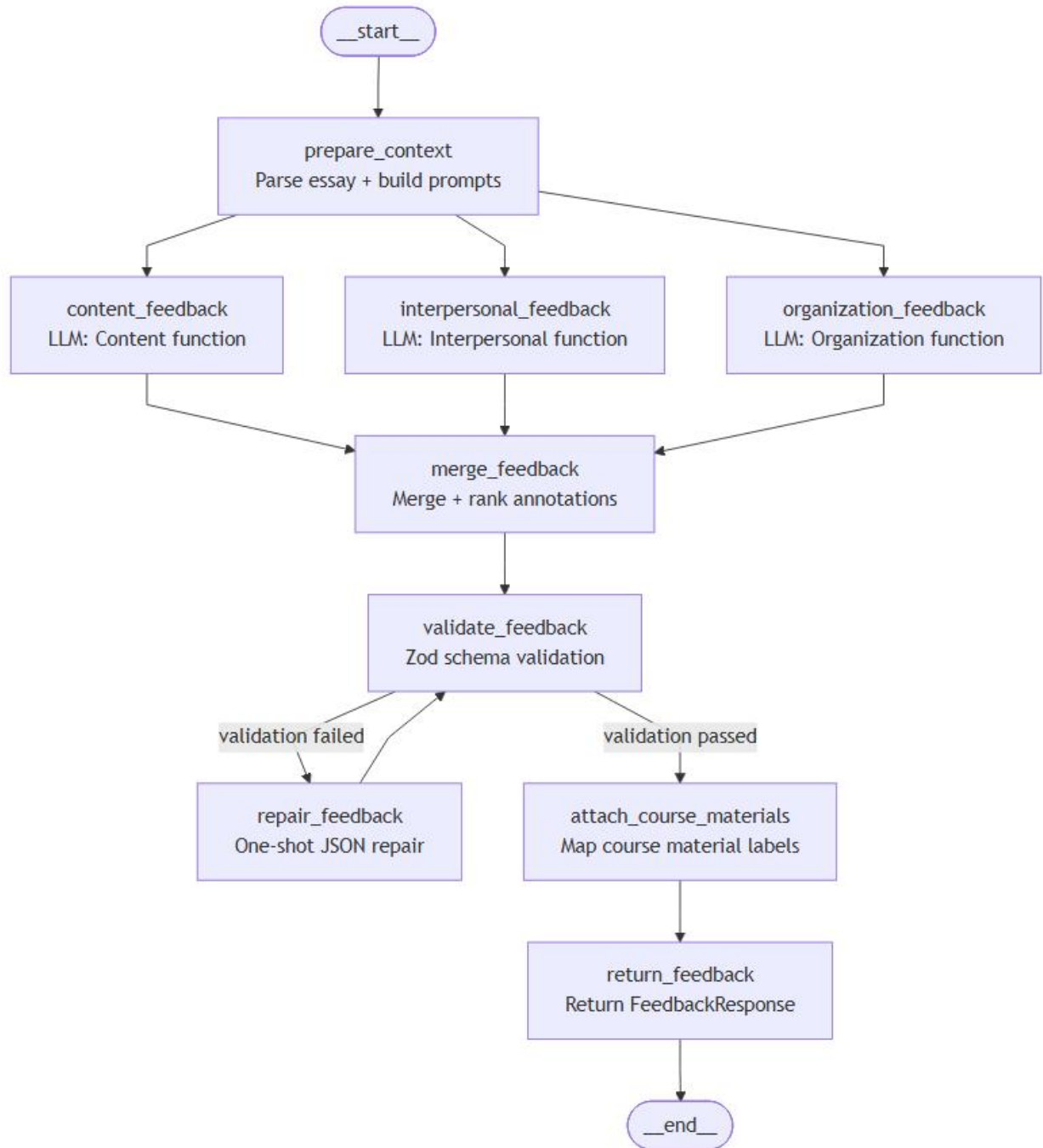
1. Workflow and Architecture

1.1 End-to-End Workflow

Stage	Behavior
Input	Paste plain text or upload .txt, .md, .doc, .docx, or .pdf; fileExtractor extracts essay_text.
Submit	POST /api/essay-feedback with body { essay_text }.
Orchestration	runFeedbackGraph: parse paragraphs, load prompt.md, run three parallel dimension branches, merge (up to 12 annotations), validate with Zod, optionally repair JSON once, attach course material labels.
Output	FeedbackResponse with paragraphs, annotations (offsets, function, level, evidence, revision guidance), and overall_feedback.
UI	Highlights linked to sidebar cards, dimension/level filters, Summary tab, sessionStorage cache for the latest result.

1.2 LangGraph Feedback Workflow (Implemented)

The diagram below reflects the StateGraph compiled in src/backend/lib/feedbackGraph.ts (@langchain/langgraph).



Graph nodes (in execution order):

- prepare_context — parseEssay + buildPromptMessages from prompt.md
- content_feedback — parallel LLM branch (Content function, max 4 annotations)
- interpersonal_feedback — parallel LLM branch (Interpersonal function)
- organization_feedback — parallel LLM branch (Organization function)
- merge_feedback — merge branches, sort by severity, cap at 12 annotations

- `validate_feedback` — Zod validation of offsets, paragraph IDs, and schema
- `repair_feedback` (conditional) — one repair attempt if validation fails
- `attach_course_materials` — map function + level to course material labels
- `return_feedback` — return `FeedbackResponse` to the API layer

Typical LLM calls per submission: about 3 (parallel dimension nodes). If repair runs, add 1 additional call.

1.3 Layered Architecture

Layer	Location / role
Presentation	<code>src/frontend/</code> — HTML, CSS, TypeScript
API	<code>src/backend/server.ts</code> — Express 5
Orchestration	<code>src/backend/lib/feedbackGraph.ts</code> — LangGraph StateGraph
LLM + Prompt	<code>openaiClient.ts</code> , <code>promptBuilder.ts</code> , <code>prompts/prompt.md</code>
Validation + Shared	<code>schemaValidator.ts</code> (Zod), <code>src/shared/schema.ts</code>
Course mapping	<code>src/shared/courseMaterials.ts</code>

1.4 Evaluation Framework

Feedback aligns with the Academic Writing Matrix: three functional dimensions (Content, Interpersonal, Organization) crossed with three levels (text, section, clause_word). The prompt forbids scoring and sentence rewriting; it provides diagnostic feedback and revision guidance only.

2. Programming Languages and Stack

Category	Choice
Primary language	TypeScript (strict: true) across backend, frontend, shared
Markup & styling	HTML5 and CSS (no React/Vue SPA framework)
Runtime	Node.js 20 or newer
Backend	Express 5
Workflow engine	LangGraph (@langchain/langgraph)
Validation	Zod 4
Document parsing	mammoth, pdf-parse, word-extractor
Uploads	multer (in-memory, max 10 MB per file)
Build	tsc for frontend bundle; tsx watch + concurrently in dev

3. LLM Models

PEAP-AI currently uses OpenAI models as the primary integration path.

The architecture is adapter-based: additional large language models (e.g., Anthropic, Google, Azure OpenAI, or other OpenAI-compatible endpoints) can be wired in by implementing the same structured-feedback contract behind the LangGraph nodes, without changing the frontend schema.

Item	Details
Primary provider	OpenAI (current production path)
Default model	gpt-5.4-mini (overridable via OPENAI_MODEL; see .env.example)
Configuration	OPENAI_API_KEY (required), OPENAI_MODEL (optional), PORT
Invocation pattern	Parallel dimension nodes + optional single repair pass
Extensibility	Other LLMs supported in principle via adapter swap or multi-provider routing

Prepared by: Lin Manshan
Email: mlin33@student.ubc.ca